

Weight — Tool Build (mechanics)

A calculator with one number key. A number costs as many taps as its value. Everyday Tools — Fordham Wong

Live link	fwong29.github.io/reusable-studio-engine/everyday-tools/
Repository	github.com/fwong29/reusable-studio-engine — project in <code>everyday-tools/</code>
Recording	everyday-tools/process/demo.mp4
README	everyday-tools/README.md — reproduced below

BACK-END ARCHITECTURE

What data does this tool need?

Three values: the current tally (how many taps toward the number being entered), the committed expression (a list of numbers and operators), and a flag for whether the current tally has been touched since the last commit. Nothing else.

Where is it stored?

In three JavaScript variables in `main.js` (`currentTally`, `tokens`, `tallyTouched`). Browser memory only.

Is it temporary or persistent?

Temporary. A page reload erases everything. There is no `localStorage`, no cookie, no database.

Does the system need memory between sessions?

No — and it must not have any. The Tool Intent's Dignity Clause says the tool records nothing; the Refusal Clause says the friction never changes with use. Both are only true if the tool forgets. Statelessness is the design, not a shortcut.

Does the system require AI inference?

No. The rule is fixed and mechanical: one tap adds one to the tally. There is nothing to infer.

How many API calls are realistically required?

Zero. The page is three static files. The only network request is a Google Fonts stylesheet, and the page works without it.

What happens if the API fails?

There is no API to fail. The one failure the system can produce is arithmetic — dividing by zero — and it renders the word `Error` in the display, which is deliberately distinct from a broken or blank state. A failure and a crash should never look the same.

LAYERS

LAYER	WHERE	WHAT
Input	<code>index.html</code> buttons → <code>click</code> listeners in <code>main.js</code>	The tally key, four operators, <code>=</code> , <code>C</code> , <code>⌫</code>
Logic	<code>pressTally</code> , <code>pressOperator</code> , <code>pressEquals</code> , <code>pressClear</code> , <code>pressBackspace</code> , <code>evaluate</code>	Increment the tally; commit it as a number when an operator or <code>=</code> is pressed; evaluate with standard precedence
Output	<code>render</code> → <code>#display</code> , <code>#tallyCount</code> , <code>#tallyMarks</code>	The running expression, the current tally as a numeral, and one mark per tap

The logic is in plain functions with no framework. Every state change goes through `render()`, so what the display shows is always exactly what the variables hold.

CHOICE POINT

The tally key is the choice point. A normal keypad makes the decision for you — one tap, digit entered. Here the decision is spread across every tap: each one is a small, visible act of continuing. The tally marks under the numeral are the seam — they show exactly how far you've committed, so the key never looks broken while a number is still being built.

BEHAVIOR INTEGRITY CHECK

- **Does it interrupt where the intent claimed?** Yes — during the behavior, at the moment a quantity becomes a number. Nothing before (no priming), nothing after (no reflection screen, no summary).
- **Does it avoid shame, surveillance, or manipulation?** It stores nothing, knows nothing about what the number means, and never comments on it. The cost of entering 7 is identical for every person, every time.
- **Is it exploiting friction or designing it?** The friction is proportional to the number and to nothing else. It doesn't escalate, doesn't personalize, doesn't reward. It's the same rule on the first use and the thousandth.
- **Is it minimal?** One number key, four operators, three controls. The decimal point was removed because it doesn't fit a tally. Nothing else was added.

ON THE ADDICTION FRAME

Leaving it unnamed is deliberate: naming a specific behavior would require the tool to know what you're counting, and refusing to know is the dignity position. The frame is present as structure — the tenth costs what the first costs, made felt — not as subject matter. The one-key build is literally a tally counter, the instrument people already use to count drinks, cigarettes, checks, reps. It just doesn't ask which.

COMMIT HISTORY

Intent statement → mechanics → ritual sketch exports → README and recording. Each is its own commit in `reusable-studio-engine`.